System Architecture & Detailed User Flows

This document outlines the detailed user flows, backend processes, and SQL interactions for the **Student Placement Attachment System**. It is designed to be your definitive cheat sheet for your final defence, detailing exactly how the View, Controller, Model, and Database interact across every single feature in the system.

Architecture Overview (MVC Pattern)

The system strictly follows the **Model-View-Controller (MVC)** architectural pattern:

1. **View:** The user interface (HTML/CSS/PHP). The user interacts with this (e.g., clicks a button or submits a form).

2. **Router:** (app/Core/Router.php) intercepts the URL and sends the request to the appropriate Controller method.

3. **Controller:** (e.g., StudentController.php) receives the request, sanitizes input, enforces role-based access via session ($this->requireAuth()), and interfaces with the Model.

4. **Model:** (e.g., Student.php) executes the SQL queries against the MySQL Database.

5. **Database:** Returns data to the Model, which passes it to the Controller, which then loads the View to display the result to the user.

---

1. Authentication & Security Flows

1.1 User Login

- **Trigger:** User submits username and password on the login page.
- **View:** app/Views/auth/login.php (Submits to /auth/login)
- **Controller:** AuthController::login()
- **Model:** User::findUserByUsername($username), User::verifyAndMigratePassword()
- **SQL Query:**SELECT * FROM users WHERE Username = ?
- **Process:** Controller fetches user. If password matches, $_SESSION is populated with UserID, Role, Username, and last_activity (for the 10-minute timeout). It checks the Role and queries specific profile models (Student/Lecturer/Host) to set role-specific session variables (e.g., $_SESSION['student_id']).

1.2 Password Reset Request

- **Trigger:** User forgets password and submits email.
- **Controller:** `AuthController::forgotPassword()`
- **Model:** `User::findUserByEmail()`, `User::savePasswordResetToken()`
- **SQL Queries:**SELECT s.UserID, CONCAT(s.FirstName, ' ', s.LastName) as Name FROM student s WHERE s.Email = ?
  UPDATE users SET ResetToken = ?, ResetTokenExpiry = ? WHERE UserID = ?
- **Process:** A unique secure token is generated and stored with an expiry of 1 hour. `Mailer::sendPasswordResetLink()` emails the user. When clicked, `AuthController::resetPassword()` validates the token and `User::updatePassword()` hashes and saves the new password.

---

2. Student Flows

2.1 First-Time Profile Completion

- **Trigger:** First login with default credentials redirects student to complete their profile.
- **Controller:** `StudentController::completeProfile()`
- **Model:** `Student::completeProfile()`
- **SQL Query:**UPDATE student SET FirstName=?, LastName=?, Email=?, PhoneNumber=?, Course=?, Faculty=?, YearOfStudy=?, EligibilityStatus='Eligible' WHERE StudentID=?
- **Process:** Updates contact details and changes their state from 'Pending' to 'Eligible', allowing them to view and apply for opportunities.

2.2 Applying for an Opportunity

- **Trigger:** Student clicks "Apply" on an active opportunity.
- **Controller:** `OpportunityController::apply()`
- **Model:** `Opportunity::apply($studentId, $opportunityId)`
- **SQL Query:** INSERT INTO jobapplication (StudentID, OpportunityID, ApplicationDate, Status) VALUES (?, ?, CURDATE(), 'Pending')
- **Process:** The model ensures no duplicate application. Records the application as 'Pending' awaiting Host Organization review.

2.3 Submitting a Weekly Logbook

- **Trigger:** Student fills out weekly tasks and submits.
- **Controller:** `LogbookController::createEntry()`
- **Model:** `Logbook::createEntry($data)`
- **SQL Query:**INSERT INTO logbook (AttachmentID, WeekNumber, StartDate, EndDate, Description, Status) VALUES (?, ?, ?, ?, ?, 'Pending')
- **Process:** JSON payload containing daily tasks is saved. Defaults to 'Pending' until reviewed by Host or Supervisor.

2.4 Uploading the Final Report

- **Trigger:** Student uploads final PDF report.
- **Controller:** `ReportController::upload()`
- **Model:** `Report::uploadFinalReport($attachmentId, $fileName, $studentId)`
- **SQL Queries:**INSERT INTO finalreport (AttachmentID, ReportPath, Status) VALUES (?, ?, 'Pending');
  UPDATE attachment SET ReportStatus = 'Submitted' WHERE AttachmentID = ?;

---

3. Host Organization Flows

3.1 Posting an Attachment Opportunity

- **Trigger:** Host posts a new vacancy.
- **Controller:** `OpportunityController::create()`
- **Model:** `Opportunity::create($data)`
- **SQL Query:**INSERT INTO attachmentopportunity (HostOrgID, Title, Description, RequiredSkills, PositionsAvailable, ApplicationStartDate, ApplicationEndDate, Status) VALUES (?, ?, ?, ?, ?, ?, ?, 'Active')

3.2 Accepting a Student Application

- **Trigger:** Host reviews applications and clicks "Accept".
- **Controller:** `OpportunityController::acceptStudent()`
- **Model:** `Opportunity::acceptStudent($applicationId, $opportunityId)`

- **SQL Queries (Transaction):**UPDATE jobapplication SET Status = 'Accepted' WHERE ApplicationID = ?;
  INSERT INTO attachment (StudentID, HostOrgID, StartDate, AttachmentStatus, ClearanceStatus) VALUES (?, ?, CURDATE(), 'Ongoing', 'Pending Clearance');
  UPDATE attachmentopportunity SET PositionsAvailable = PositionsAvailable - 1 WHERE OpportunityID = ?;

3.3 Adding Logbook Remarks

- **Trigger:** Host supervisor reads a student's weekly logbook and adds a comment.
- **Controller:** `LogbookController::addComment()`
- **Model:** `Logbook::updateHostComment($logbookId, $comment)`
- **SQL Query:**UPDATE logbook SET HostSupervisorComments = ?, Status = 'Reviewed' WHERE LogbookID = ?

3.4 Verifying the Final Report

- **Trigger:** Host views the student's final report and clicks "Approve".
- **Controller:** `HostController::verifyReport()`
- **Model:** `Report::updateFinalReportStatus($attachmentId, 'Approved')`
- **SQL Queries (Transaction):**UPDATE finalreport SET Status = 'Approved' WHERE AttachmentID = ?;
  UPDATE attachment SET AttachmentStatus = 'Completed', ClearanceStatus = 'Cleared' WHERE AttachmentID = ?;
  UPDATE student s JOIN attachment a ON s.StudentID = a.StudentID SET s.EligibilityStatus = 'Cleared' WHERE a.AttachmentID = ?;
- **Process:** Approval immediately completes the attachment and synchronizes the student's global eligibility to "Cleared".

---

4. Lecturer (Academic Supervisor) Flows

4.1 Generating an Assessment Code

- **Trigger:** Lecturer generates a 6-digit verification code before assessing the student.
- **Controller:** `HostController::generateCode()`
- **Model:** `Assessment::generateVerificationCode($attachmentId)`

- **SQL Query:**UPDATE attachment SET VerificationCode = ?, CodeExpiry = ? WHERE AttachmentID = ?

4.2 Conducting an Assessment

- **Trigger:** Lecturer submits the assessment form with marks and the verification code.
- **Controller:** `AssessmentController::conduct()`
- **Model:** `Assessment::create($data)`
- **SQL Queries:**INSERT INTO assessment (AttachmentID, LecturerID, AssessmentType, AssessmentDate, Marks, Remarks, Status) VALUES (?, ?, ?, ?, ?, ?, 'Completed');
- **Process:** Controller verifies the code (`Assessment::verifyCode()`). Validates that `AssessmentType` (1st vs 2nd) hasn't been duplicated for this student by this lecturer. Final marks calculate combined scores if both assessments are present.

4.3 Approving Logbooks

- **Trigger:** Lecturer reviews logbooks and marks them "Approved".
- **Controller:** `LogbookController::approve()`
- **Model:** `Logbook::approve($logbookId, $lecturerId)`
- **SQL Query:**UPDATE logbook SET Status = 'Approved', ReviewedBy = ?, ReviewedAt = NOW() WHERE LogbookID = ?

---

5. Admin (Industrial Attachment Coordinator) Flows

5.1 Single and Bulk User Creation

- **Trigger:** Admin uploads a CSV to create Students, Supervisors, or Hosts.
- **Controller:** `AdminController::bulkUploadStudents()`, `StaffController::bulkUploadSupervisors()`
- **Model:** `Student::createFromAdmin()`, `Supervisor::createBulk()`
- **SQL Queries (Transaction loop):**INSERT INTO users (Username, Password, Role, Status) VALUES (?, ?, 'Student', 'Active');
  INSERT INTO student (UserID, FirstName, LastName, Faculty, Department, EligibilityStatus) VALUES (?, ?, ?, ?, ?, 'Pending');

5.2 Bulk Assigning Supervisors

- **Trigger:** Admin assigns multiple lecturers to multiple students.
- **Controller:** `BulkSupervisionController::processAssignment()`
- **Model:** `Supervisor::assign($attachmentId, $lecturerId)`
- **SQL Query:**INSERT INTO supervision (AttachmentID, LecturerID) VALUES (?, ?)
- **Process:** System guarantees max 2 supervisors per attachment and requires the first assessment to be complete before assigning a second supervisor. Emails are queued to notify lecturers.

5.3 Manual Student Clearance

- **Trigger:** Admin clicks "Force Complete & Clear" on the Student Progress dashboard.
- **Controller:** `AdminController::completeAttachment()`
- **SQL Queries (Transaction):**UPDATE attachment SET AttachmentStatus = 'Completed', ClearanceStatus = 'Cleared' WHERE AttachmentID = ?;
  UPDATE student SET EligibilityStatus = 'Cleared' WHERE StudentID = ?;

5.4 System Settings Management

- **Trigger:** Admin toggles settings like `require_host_approval` or sets max attachment duration.
- **Controller:** `SettingsController::update()`
- **Model:** `Admin::updateSettings($settingsArray)`
- **SQL Query:**UPDATE system_settings SET SettingValue = ? WHERE SettingKey = ?

---

6. Automated Background Tasks (Cron Jobs)

6.1 Assessment Email Alerts (< 24 Hours)

- **Trigger:** A server cron job executes `cron/assessment_alerts.php` daily.
- **Controller/Script:** `cron/assessment_alerts.php`
- **Model/Helper:** Custom query relying on `Mailer::notifyAssessmentReminder()`
- **SQL Query:**SELECT a.AssessmentID, a.AssessmentType, a.AssessmentDate, a.LecturerID, att.StudentID, att.HostOrgID, s.FirstName, s.LastName, s.Email as StudentEmail, l.Name as LecturerName, l.Email as LecturerEmail, ho.OrganizationName, ho.Email as HostEmail

```
FROM assessment a
JOIN attachment att ON a.AttachmentID = att.AttachmentID
JOIN student s ON att.StudentID = s.StudentID
LEFT JOIN lecturer l ON a.LecturerID = l.LecturerID
LEFT JOIN hostorganization ho ON att.HostOrgID = ho.HostOrgID
WHERE a.Status = 'Scheduled' AND a.AssessmentDate = CURDATE() + INTERVAL 1
DAY
```

- **Process:** Evaluates scheduled assessments exactly 24 hours out. Iterates the result set and triggers unified email notifications to the Student, Lecturer, and Host Organization simultaneously.

---

7. Security Architecture & Implementations

A critical aspect of the system is how it protects data and ensures secure operations. Below are the key security features implemented and exactly where they occur in the codebase:

7.1 Cross-Site Request Forgery (CSRF) Protection

- **Purpose:** Prevents malicious websites from executing unauthorized commands on behalf of an authenticated user.
- **Where it happens:**
  - **Generation:** A secure token is generated in the session upon initialization.
  - **Views:** Every POST form includes `<input type="hidden" name="csrf_token" value="<?= $_SESSION['csrf_token'] ?? '' ?>">`.
  - **Controllers:** Enforced via `$this->verifyCsrf()` in the base `app/Core/Controller.php` before processing any `POST` request (e.g., `AuthController::login()`, `AssessmentController::conduct()`).

7.2 SQL Injection (SQLi) Prevention

- **Purpose:** Prevents attackers from manipulating database queries via malicious input.
- **Where it happens:**
  - **Models: 100%** of dynamic database queries across all Models use Prepared Statements (`$stmt->prepare()` and `$stmt->bind_param()`).
  - User input is never directly concatenated into SQL strings.

7.3 Role-Based Access Control (RBAC) & Session Timeout

- **Purpose:** Ensures users can only access endpoints authorized for their specific role, and logs them out if inactive.
- **Where it happens:**
  - **Controllers:** Every protected controller method calls `$this->requireAuth('role_name')` (located in `app/Core/Controller.php`).
  - **Timeout:** The `requireAuth` method compares the current time against `$_SESSION['last_activity']`. If the difference exceeds **10 minutes (600 seconds)**, the session is destroyed, forcing a re-login.

7.4 Secure Password Management (Hashing)

- **Purpose:** Protects user passwords if the database is ever compromised.
- **Where it happens:**
  - **Models (`User.php`):** The system uses PHP's native `password_hash()` utilizing the secure **Bcrypt** algorithm.
  - **Migration:** The `verifyAndMigratePassword()` method seamlessly handles legacy plaintext passwords. Upon a successful login with a plaintext password, it instantly hashes the password and updates the database record.

7.5 Cross-Site Scripting (XSS) Prevention

- **Purpose:** Prevents malicious scripts from being injected into the views and executed in other users' browsers.
- **Where it happens:**
  - **Views:** When displaying any user-submitted data (e.g., names, logbook descriptions, remarks), the data is wrapped in `htmlspecialchars()` to escape HTML entities.
  - **Controllers:** User input is cleaned using `Helpers::sanitize()` before processing.

7.6 Database Transactions (Data Integrity)

- **Purpose:** Guarantees atomicity. If a complex operation involving multiple tables fails halfway, all changes are rolled back to prevent orphaned or corrupted data.
- **Where it happens:**
  - **Models:** Uses `$this->conn->begin_transaction()`, `commit()`, and `rollback()`.
  - **Examples:** `Opportunity::acceptStudent()` (updates application status AND creates attachment), `Report::updateFinalReportStatus()` (approves report AND completes attachment AND clears student).